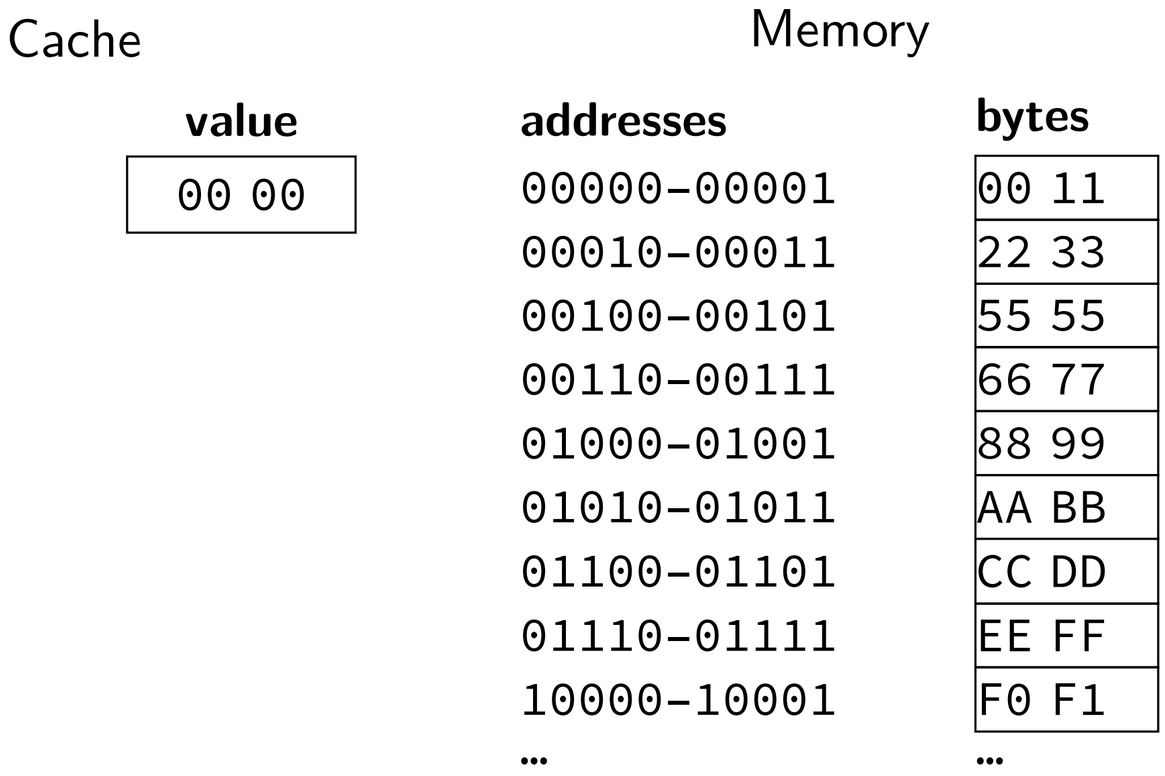
one-block cache

one-block cache



one-block cache

decision: divide memory into two-byte blocks
put exactly one of these blocks in the cache

Cache

value
00 00

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA BB
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

one-block cache

read byte at 01011?

Cache

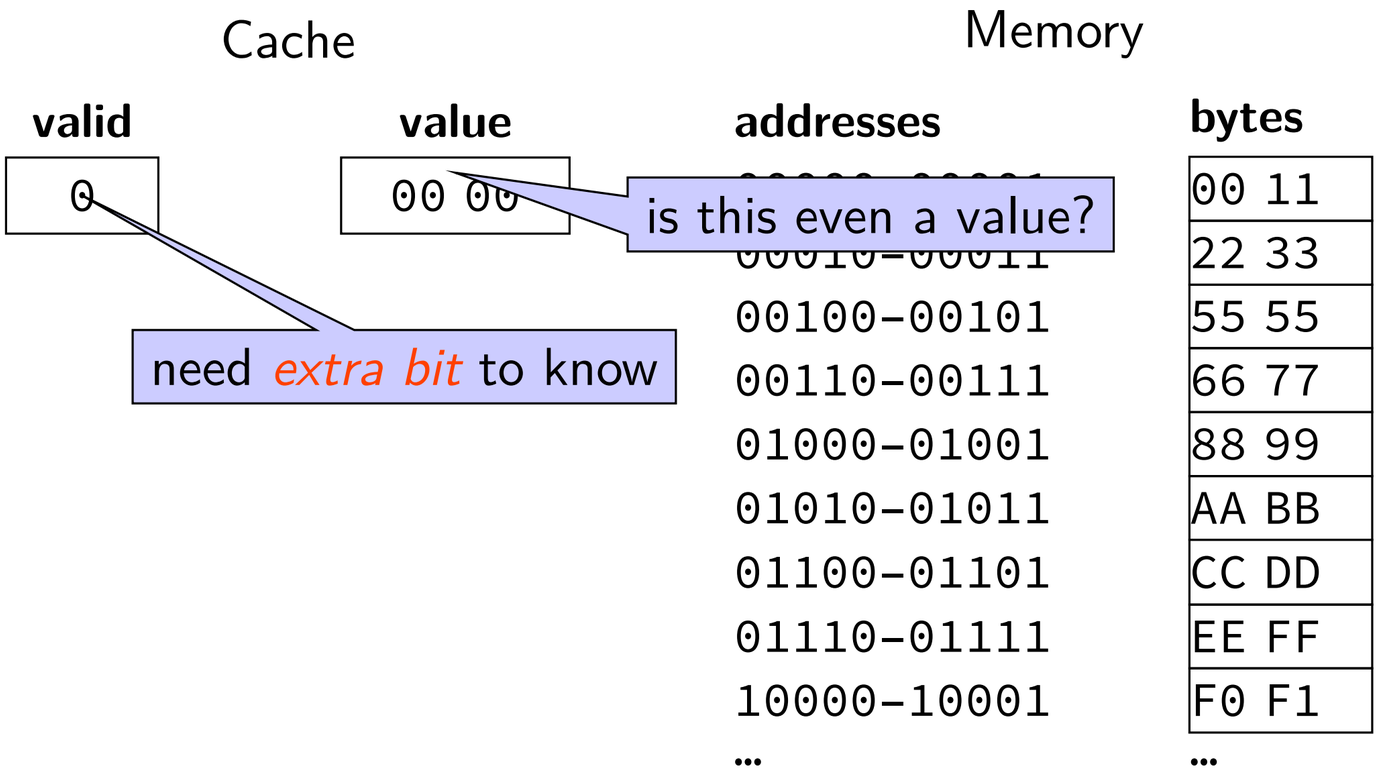
value
00 00

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA BB
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

one-block cache

read byte at 01011?



one-block cache

read byte at 01011?
invalid, fetch

Cache

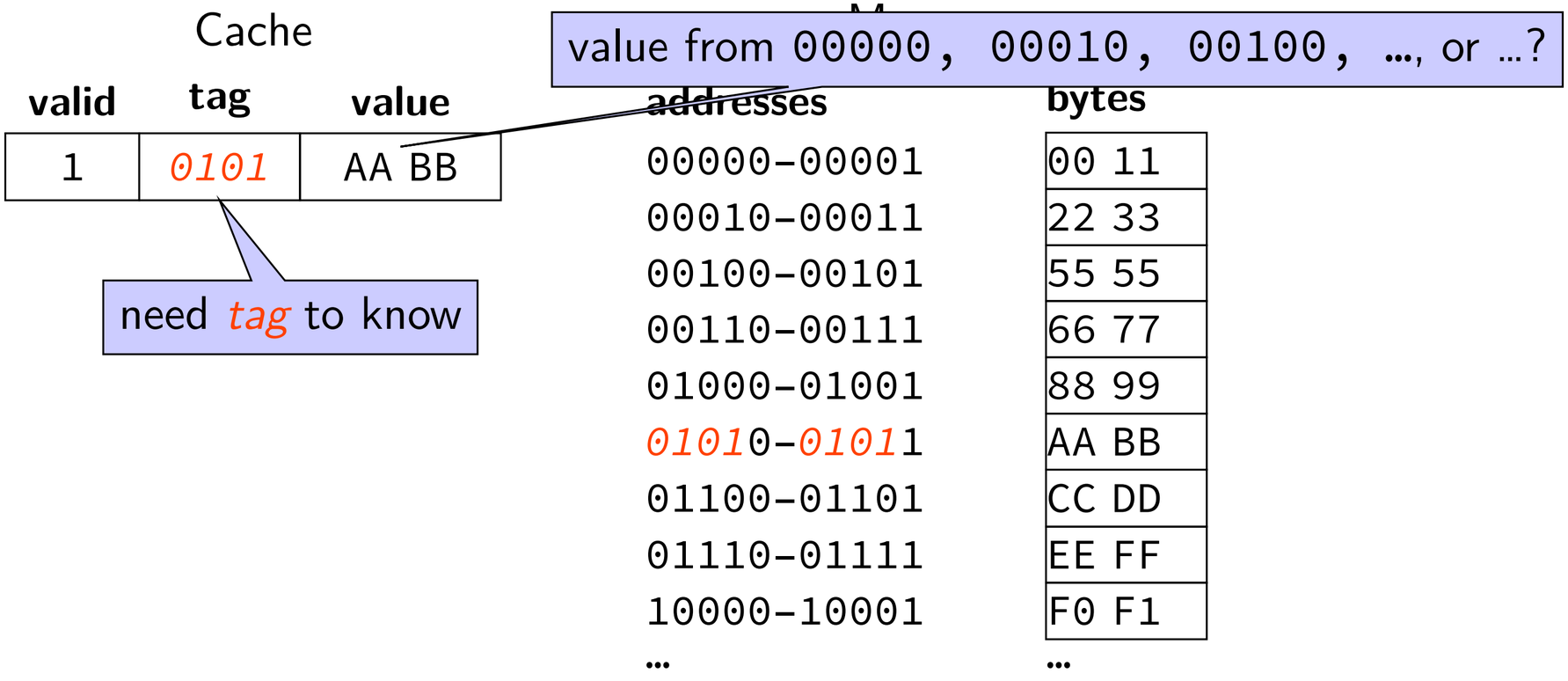
valid	value
1	*AA BB*

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA BB
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

one-block cache

read byte at *0101*1?



one-block cache

read byte at 01011*1*?

Cache

valid	tag	value
1	0101	AA *BB*

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA *BB*
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

building a (direct-mapped) cache

building a (direct-mapped) cache

Cache

value
00 00
00 00
00 00
00 00

cache block: *2 bytes*

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA BB
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

building a (direct-mapped) cache

read byte at 01011?

Cache

value
00 00
00 00
00 00
00 00

cache block: 2 bytes

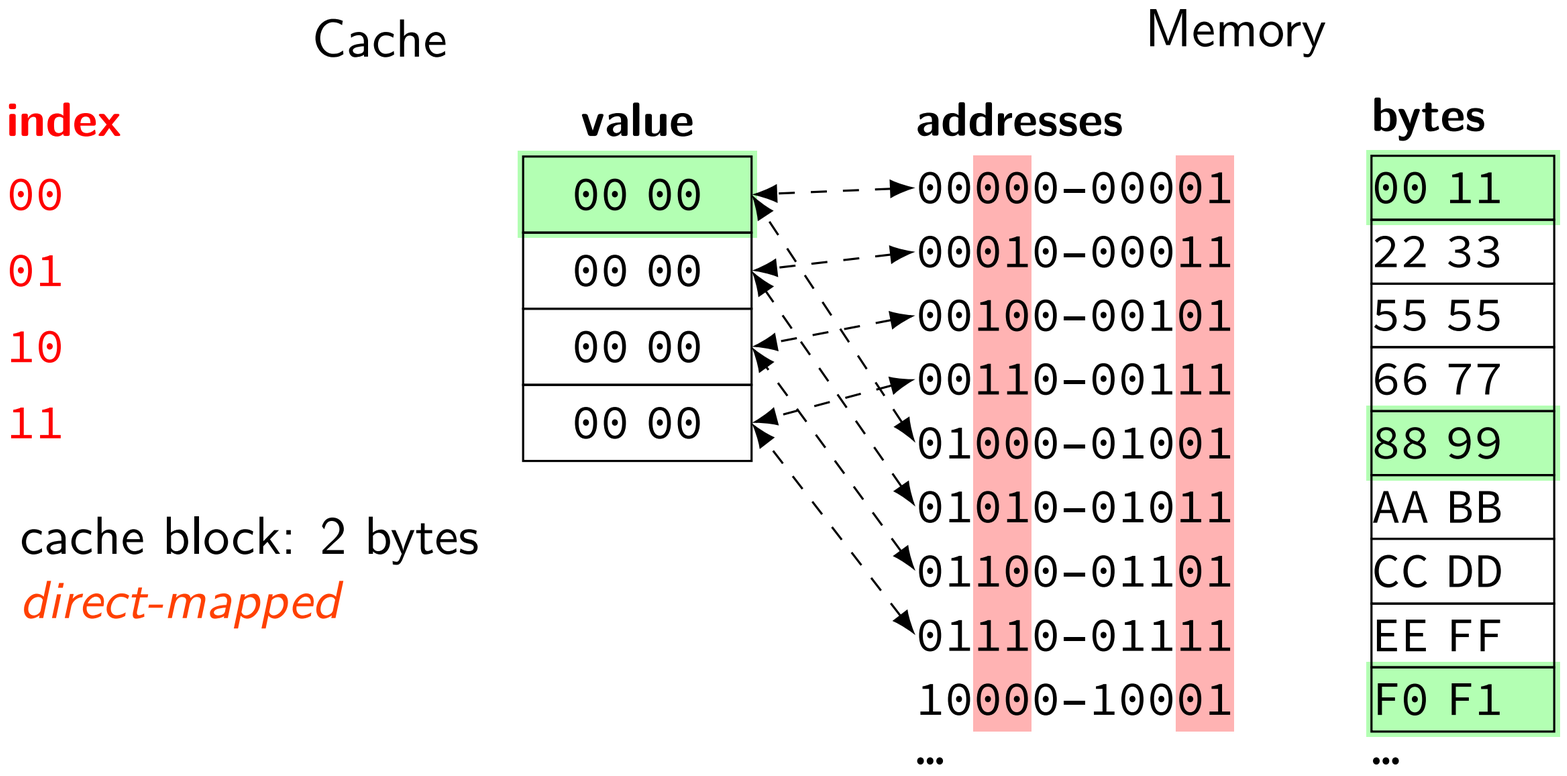
Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA BB
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

building a (direct-mapped) cache

read byte at 01011?

exactly *one place* for each address
spread out what can go in a block

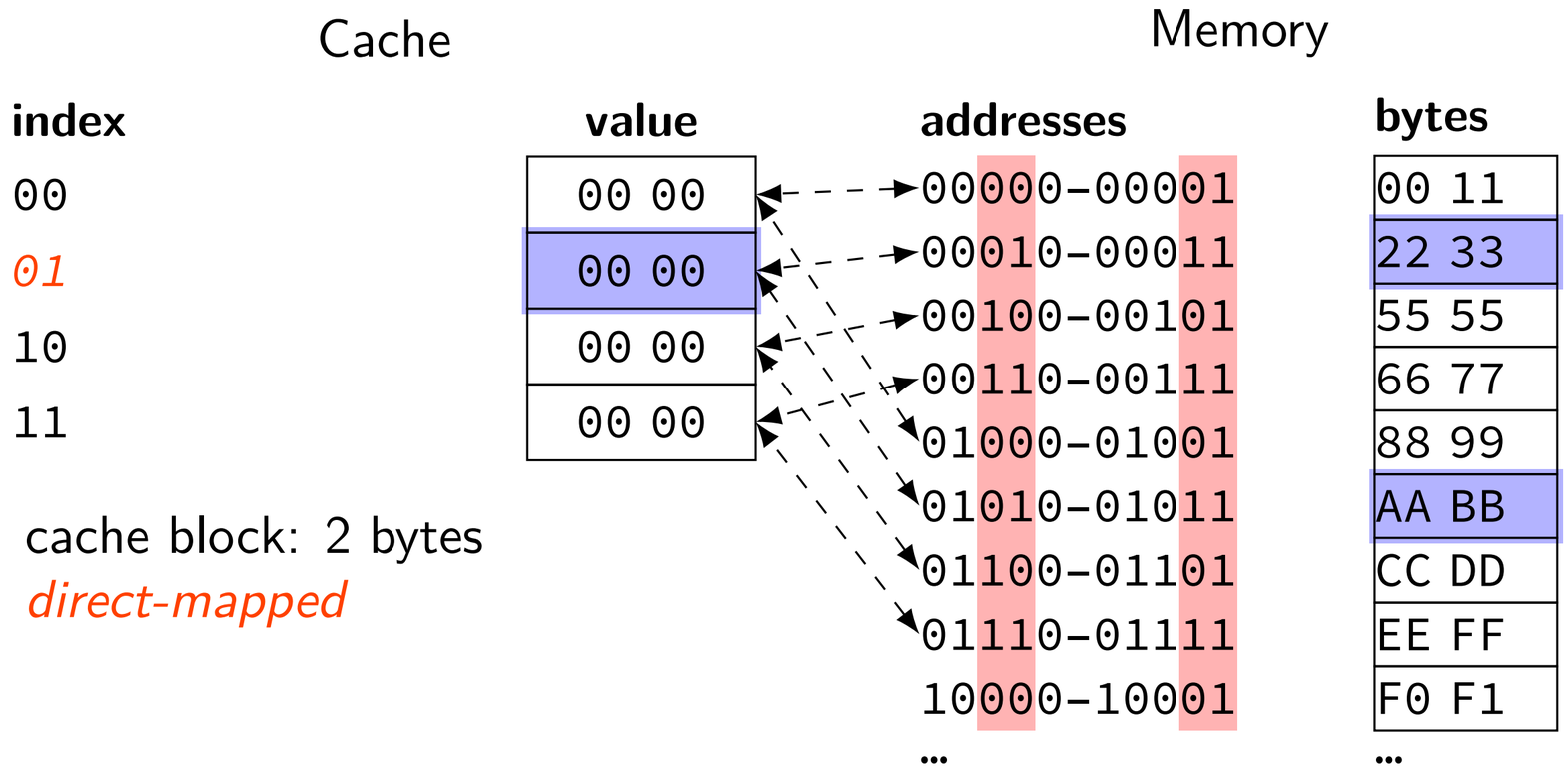


cache block: 2 bytes
direct-mapped

building a (direct-mapped) cache

read byte at 01*01*1?

exactly *one place* for each address
spread out what can go in a block

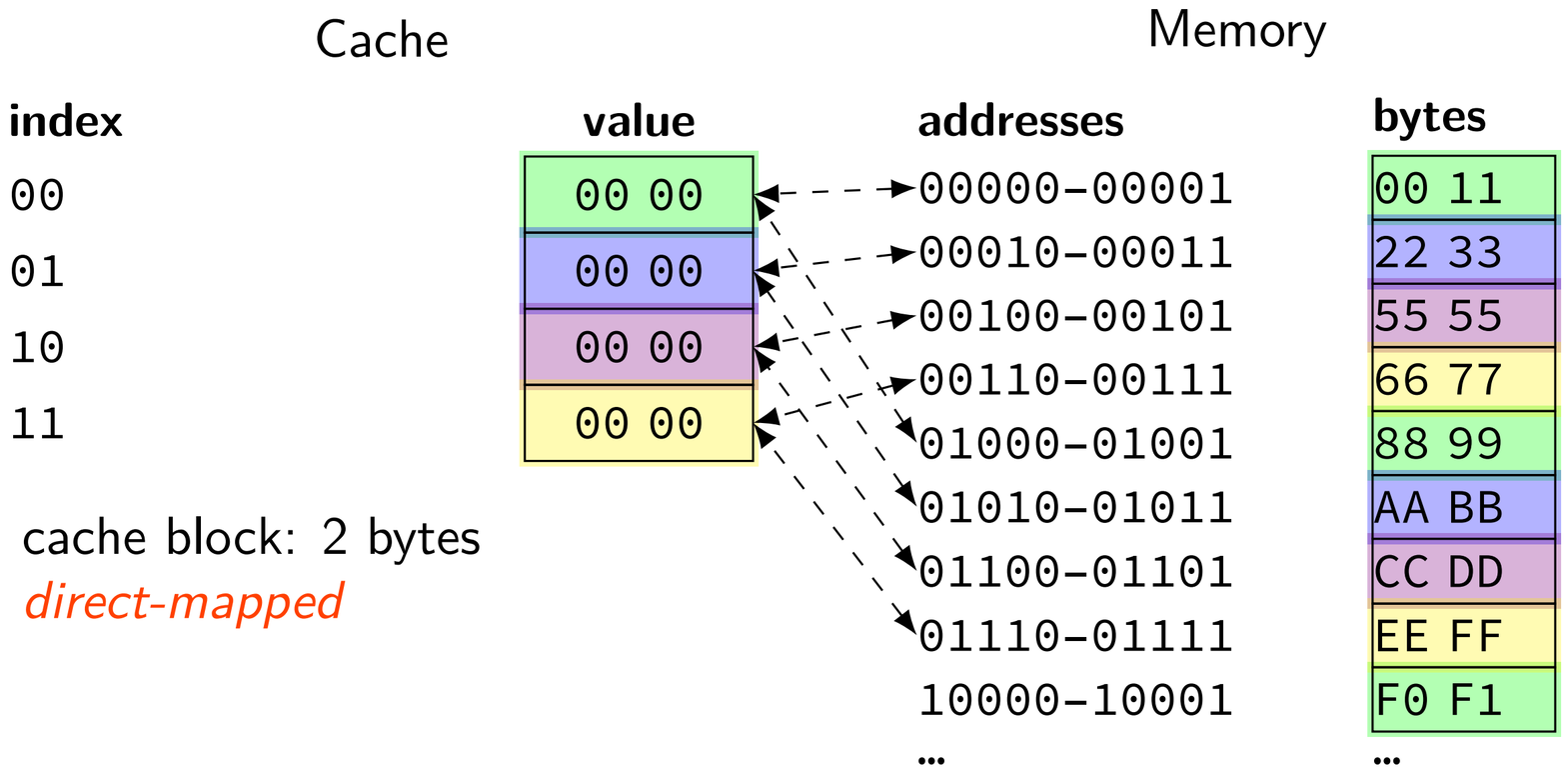


cache block: 2 bytes
direct-mapped

building a (direct-mapped) cache

read byte at 01*01*1?

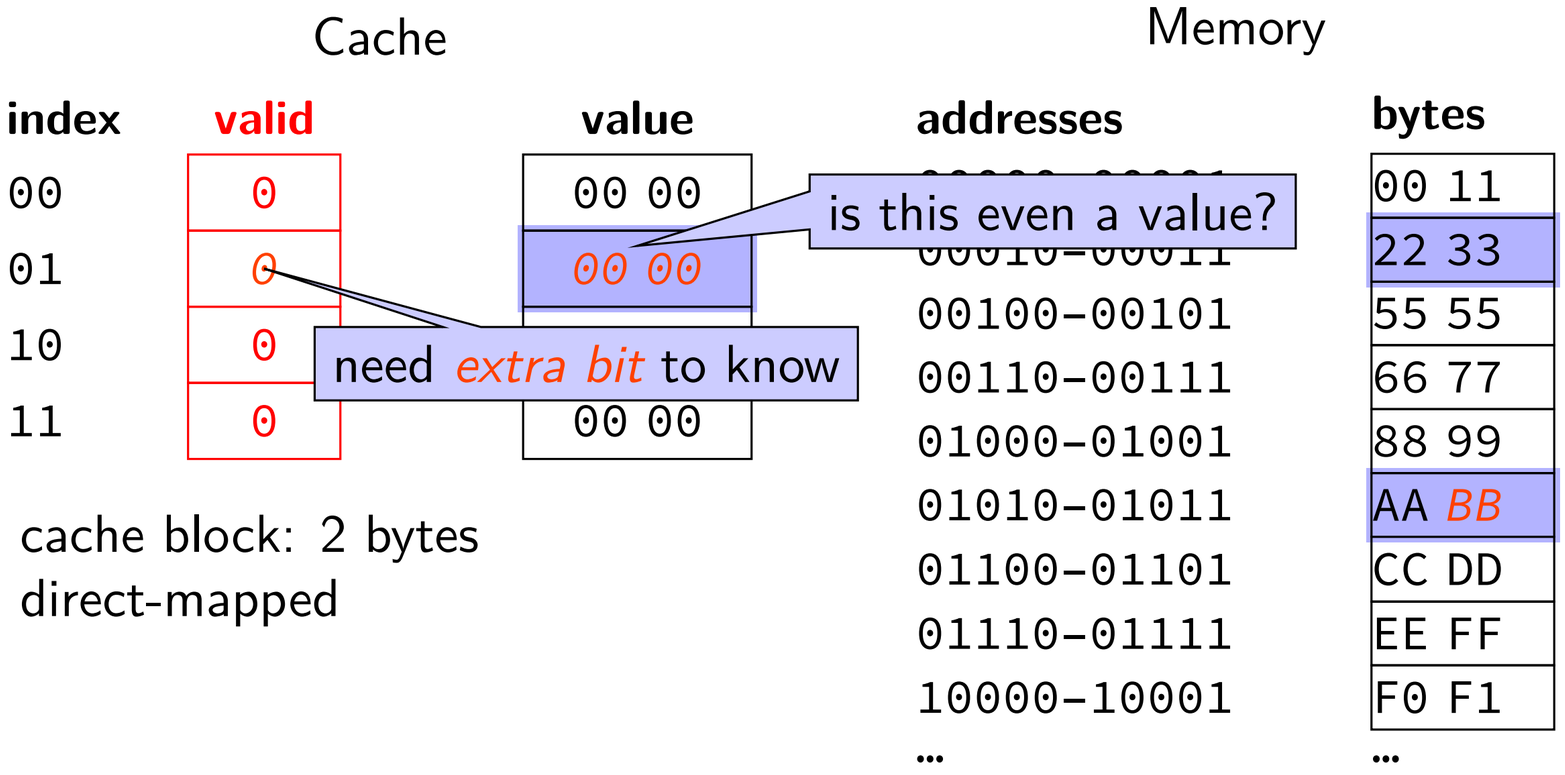
exactly *one place* for each address
spread out what can go in a block



cache block: 2 bytes
direct-mapped

building a (direct-mapped) cache

read byte at 01011?



cache block: 2 bytes
direct-mapped

building a (direct-mapped) cache

read byte at 01011?
invalid, fetch

Cache

index	valid	value
00	0	00 00
01	*1*	*AA BB*
10	0	00 00
11	0	00 00

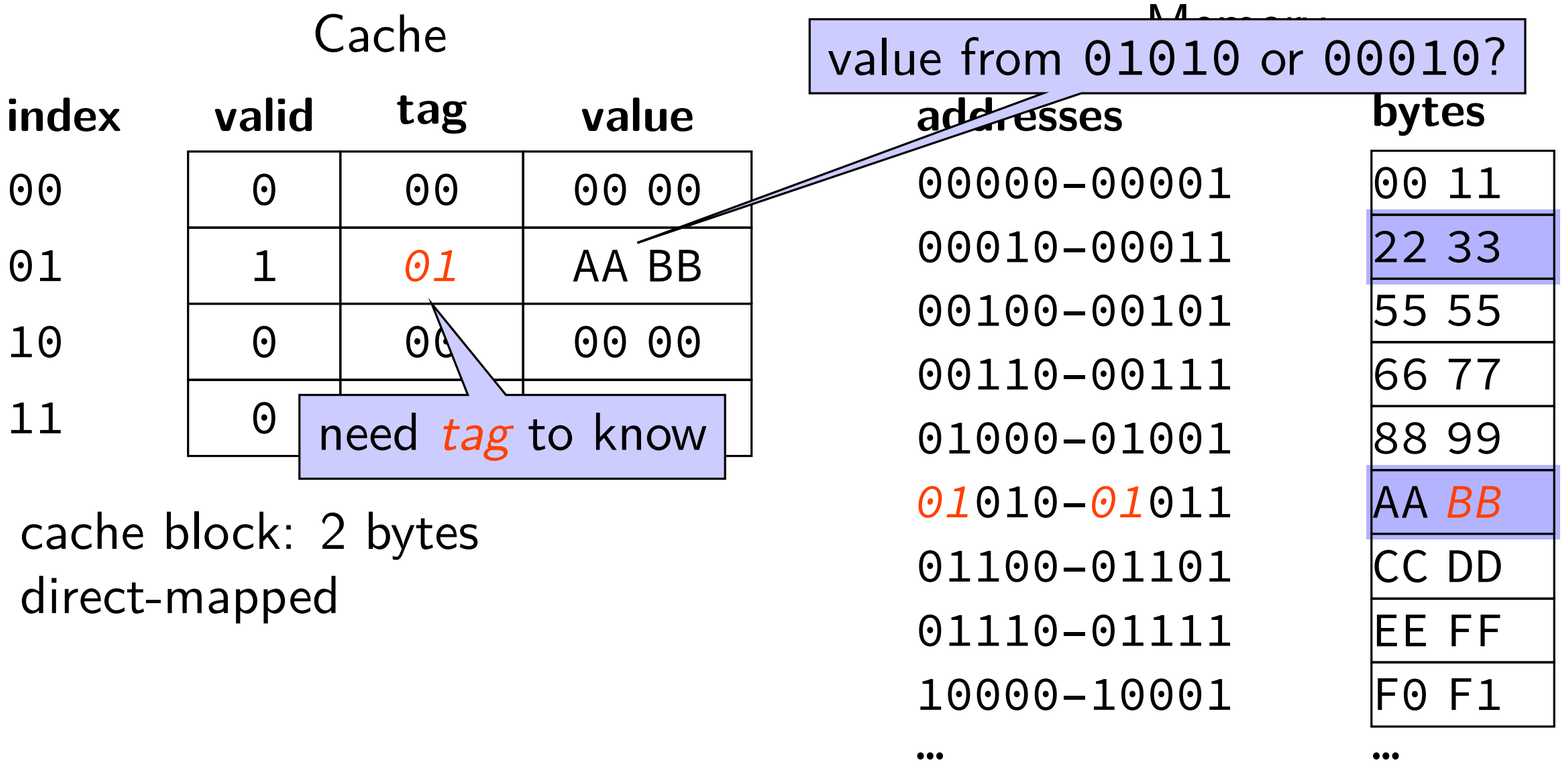
cache block: 2 bytes
direct-mapped

Memory

addresses	bytes
00000–00001	00 11
00010–00011	22 33
00100–00101	55 55
00110–00111	66 77
01000–01001	88 99
01010–01011	AA *BB*
01100–01101	CC DD
01110–01111	EE FF
10000–10001	F0 F1
…	…

building a (direct-mapped) cache

read byte at 01011?
invalid, fetch



cache block: 2 bytes
direct-mapped

terminology

row = set

preview: change how much is in a row

cache analogy: you finding words in books

block size is 1 book page

book title: tag

page number: index

word index on book page: offset

Tag-Index-Offset (TIO)

Tag-Index-Offset (TIO)

address `001111` (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets			
2 byte blocks, 8 sets			
4 byte blocks, 2 sets			

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	EE *FF*

4 byte blocks, 2 sets

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

Tag-Index-Offset (TIO)

address `00111`*1* (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets			1
2 byte blocks, 8 sets			1
4 byte blocks, 2 sets			

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	EE *FF*

$2 = 2^1$ bytes in block
1 bit to say which byte

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

4 byte blocks, 2 sets

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

Tag-Index-Offset (TIO)

address `0011`*`11`* (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets			1
2 byte blocks, 8 sets			1
4 byte blocks, 2 sets			11

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	

$4 = 2^2$ bytes in block
2 bits to say which byte

4 byte blo

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

Tag-Index-Offset (TIO)

address `001111` (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, *4 sets*		11	1
2 byte blocks, *8 sets*			1
4 byte blocks, *2 sets*		1	11

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	EE *FF*

4 byte blocks, 2 sets

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

$2^2 = 4$ sets
2 bits to index set

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001		01	F1 F2
010		--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

Tag-Index-Offset (TIO)

address `001111` (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets		11	1
2 byte blocks, 8 sets		111	1
4 byte blocks, 2 sets		1	11

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	

4 byte bl

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

$2^3 = 8$ sets

3 bits to index set

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

Tag-Index-Offset (TIO)

address `001111` (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets		11	1
2 byte blocks, 8 sets		111	1
4 byte blocks, 2 sets		1	11

2 byte blocks, 4 sets

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	EE *FF*

4 byte blocks, 2 sets

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100			-- --
101			AA BB
110	0	--	-- --
111	1	00	EE *FF*

$2^1 = 2$ sets
1 bit to index set

Tag-Index-Offset (TIO)

address `001111` (stores value `0xFF`)

cache	tag	index	offset
2 byte blocks, 4 sets	001	11	1
2 byte blocks, 8 sets	00	111	1
4 byte blocks, 2 sets	001	1	11

tag — whatever is left over

index	valid	tag	value
00	1	000	00 11
01	1	001	AA BB
10	0	--	-- --
11	1	001	EE *FF*

4 byte blocks, 2 sets

index	valid	tag	value
0	1	000	00 11 22 33
1	1	001	CC DD EE *FF*

2 byte blocks, 8 sets

index	valid	tag	value
000	1	00	00 11
001	1	01	F1 F2
010	0	--	-- --
011	0	--	-- --
100	0	--	-- --
101	1	00	AA BB
110	0	--	-- --
111	1	00	EE *FF*

cache size

cache size = amount of *data* in cache

not included metadata (tags, valid bits, etc.)

Tag-Index-Offset formulas (direct-mapped)

(formulas derivable from prior slides)

$S = 2^s$	number of sets
s	(set) index bits
$B = 2^b$	block size
b	(block) offset bits
m	memory addreses bits
$t = m - (s + b)$	tag bits
$C = B \times S$	cache size (if direct-mapped)

Tag-Index-Offset formulas (direct-mapped)

(formulas derivable from prior slides)

$S = 2^s$	number of sets
s	(set) index bits
$B = 2^b$	block size
b	(block) offset bits
m	memory addreses bits
$t = m - (s + b)$	tag bits
$C = B \times S$	*cache size* (if direct-mapped)

TIO: exercise

64-byte blocks, 128 set cache

stores $64 \times 128 = 8192$ bytes (of data)

if addresses 32-bits, then how many tag/index/offset bits?

which bytes are stored in the same block as byte from `0x1037`?

A. byte from `0x1011`
B. byte from `0x1021`
C. byte from `0x1035`
D. byte from `0x1041`

cache size exercise

A system uses a direct-mapped cache with 32 byte blocks.

Two memory accesses are made:

- Read `0x08249`
- Read `0x0c658`

What is the fewest number of cache sets this cache could have to store both values at the same time?

What is the size of this cache?

example access pattern (setup)

address (as hex)	result
00000000(00)	
00000001(01)	
01100001(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

2 byte blocks, 4 sets

index	valid	tag	value
00			
01			
10			
11			

example access pattern (setup)

address (as hex)	result
00000000(00)	
00000001(01)	
01100001(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

2 byte blocks, 4 sets

index	valid	tag	value
00			
01			
10			
11			

$B = 2 = 2^b$ byte-block size

$b = 1$ (block) offset bits

$S = 4 = 2^s$ sets

$s = 2$ (set) index bits

m = 8 bit addresss

$t = m - (s + b) = 5$ tag bits

example access pattern (steps 1-4)

address (as hex)	result
00000000(00)	
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00			
01			
10			
11			

example access pattern (steps 1-4)

address (as hex)	result
00000000(00)	*miss*
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	*1*	*00000*	*mem[0x00]* *mem[0x01]*
01			
10			
11			

example access pattern (steps 1-4)

address (as hex)	result
00000000(00)	miss
00000001(01)	*hit*
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	*00000*	mem[0x00] mem[0x01]
01			
10			
11			

example access pattern (steps 1-4)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	*miss*
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	*1*	*01100*	*mem[0x62]* *mem[0x63]*
10			
11			

access to 0x61 matches set index 00

block's tag 00000 != 0x61's tag of 01100

need to *replace* value in that block

example access pattern (steps 1-4)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	*miss*
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10			
11			

access to 0x61 matches set index 00

block's tag 00000 != 0x61's tag of 01100

need to *replace* value in that block

example access pattern (steps 4-7)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	*miss*
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	*1*	*01100*	*mem[0x60]* *mem[0x61]*
01	1	01100	mem[0x62] mem[0x63]
10			
11			

example access pattern (steps 4-7)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	01100	mem[0x60] mem[0x61]
01	1	01100	mem[0x62] mem[0x63]
10			
11			

example access pattern (steps 4-7)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	*hit*
00000000(00)	
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	01100	mem[0x60] mem[0x61]
01	*1*	*01100*	*mem[0x62]* *mem[0x63]*
10			
11			

example access pattern (steps 4-7)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	hit
00000000(00)	*miss*
01100100(64)	

tag ind

2 byte blocks, 4 sets

index	valid	tag	value
00	1	01100	mem[0x60] mem[0x61]
01	1	01100	mem[0x62] mem[0x63]
10			

access to 0x00 is miss even though:

- we had extra space in the cache to store blocks
- we previously accesses the 0x00-0x01 block

called “conflict miss” (versus “compulsory” or “capacity” miss)

aside: miss types

compulsory:

- first access to block
- avoid by increasing block size or “prefetching”

capacity:

- trying to put too much stuff in the cache
- avoid with larger cache

conflict:

- index bits happen to match up too often
- avoid with more “associative” cache (next topic)

example access pattern (steps 7-8)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	hit
00000000(00)	miss
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x00]
01	1	01100	mem[0x62] mem[0x63]
10			
11			

example access pattern (steps 7-8)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	hit
00000000(00)	*miss*
01100100(64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	*00000*	*mem[0x00]* *mem[0x00]*
01	1	01100	mem[0x62] mem[0x63]
10			

place 0x00-0x01 back in cache set 0

would have been better if never replaced…

… but cache can't see the future to know that

example access pattern (steps 7-8)

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	hit
00000000(00)	miss
01100100(64)	*miss*

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x00]
01	1	01100	mem[0x62] mem[0x63]
10	1	*01100*	*mem[0x64]* *mem[0x65]*
11			

direct-mapped cache exercise (1)

address (as hex)	result
00000000(00)	
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

4 byte blocks, 4 sets

index	valid	tag	value
00			
01			
10			
11			

exercise: how to split addresses?

solution

$4 = 2^b$ byte block size

$b = 2$ (block) offset bits

$S = 4 = 2^s$ sets

$s = 2$ (set) index bits

$m = 8$ bit addresses

$t = m - (s + b) = 4$ tag bits

direct-mapped cache exercise (2)

address (as hex)	result
00000000(00)	
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01101000(64)	

tag index offset

4 byte blocks, 4 sets

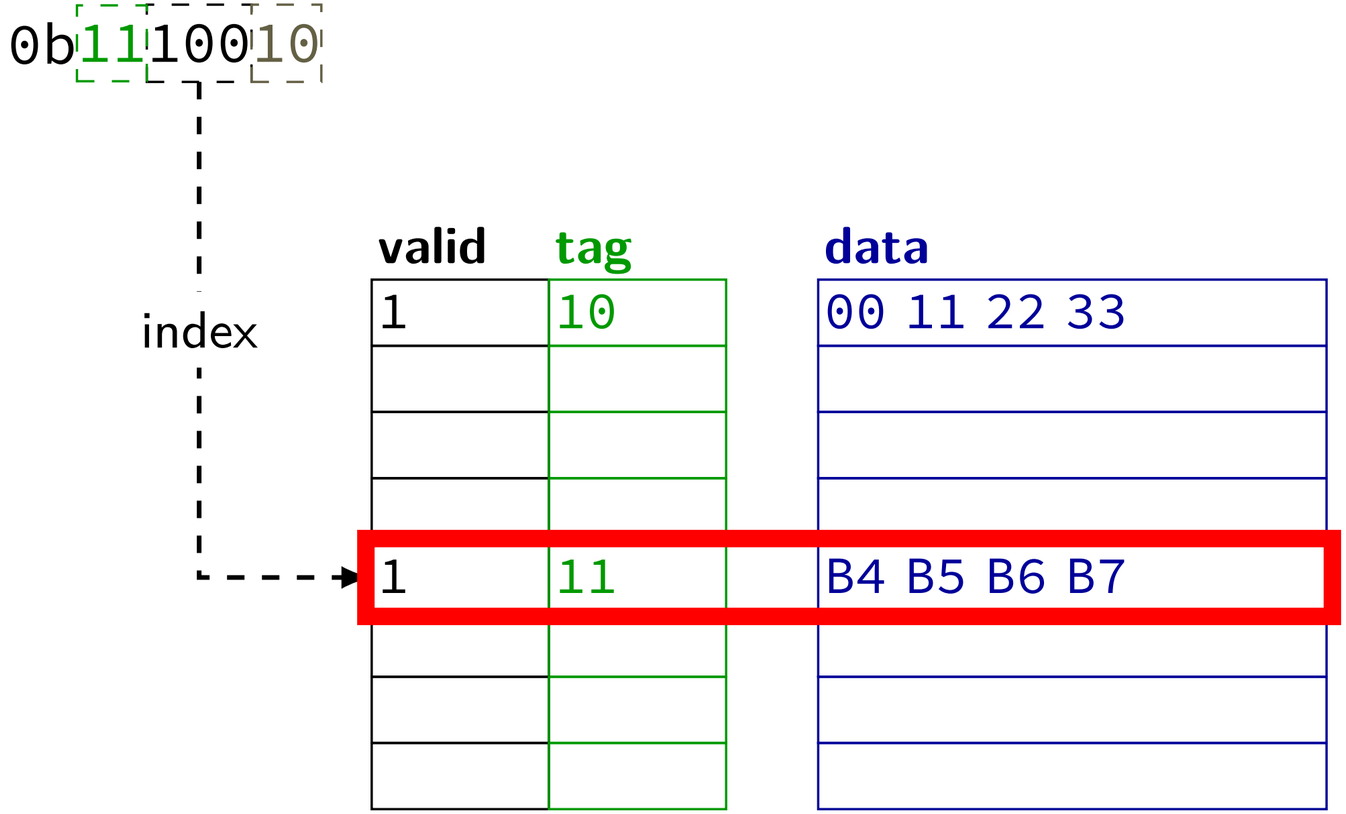
index	valid	tag	value
00			
01			
10			
11			

exercise: which accesses are hits?

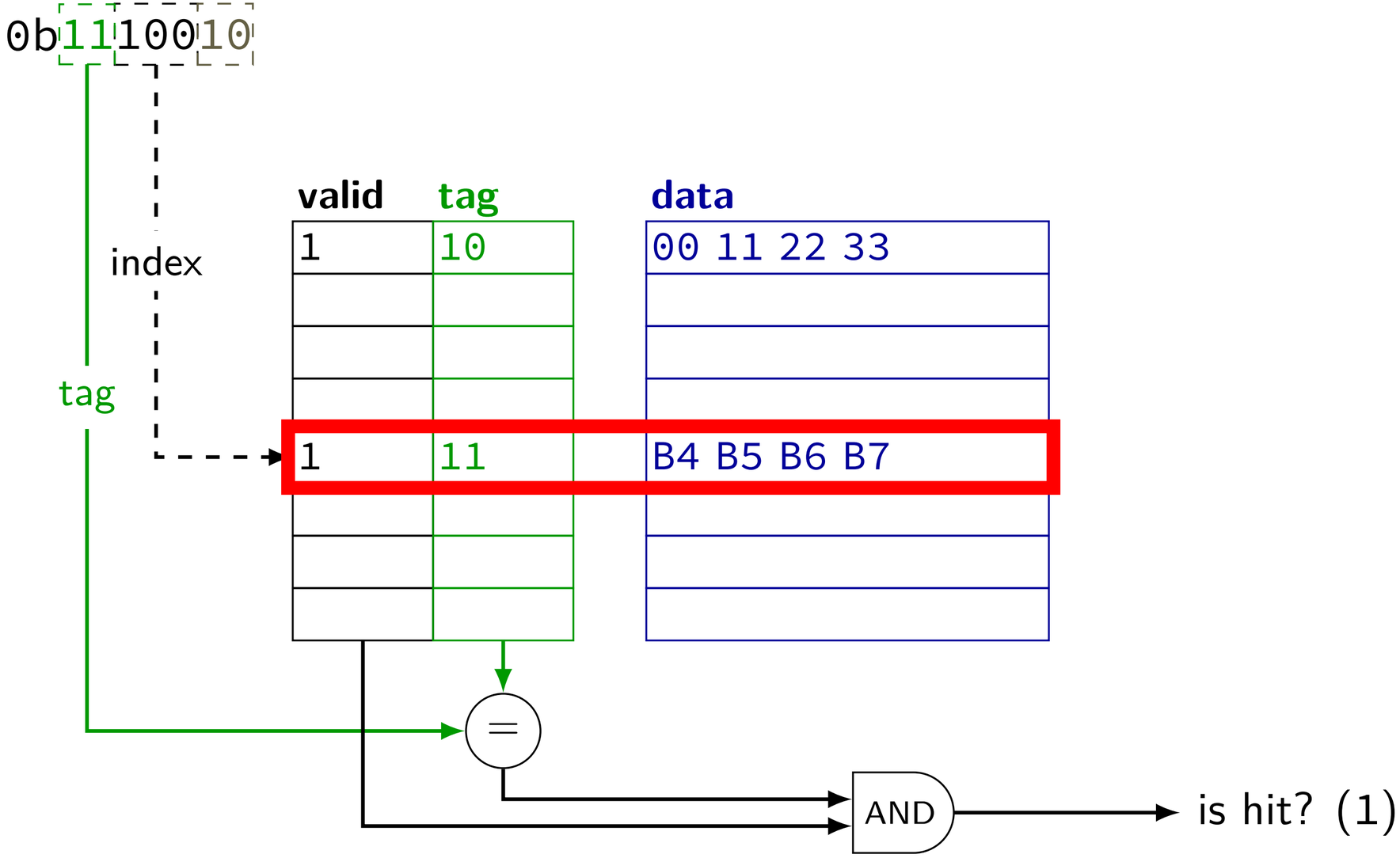
backup slides

cache operation (read)

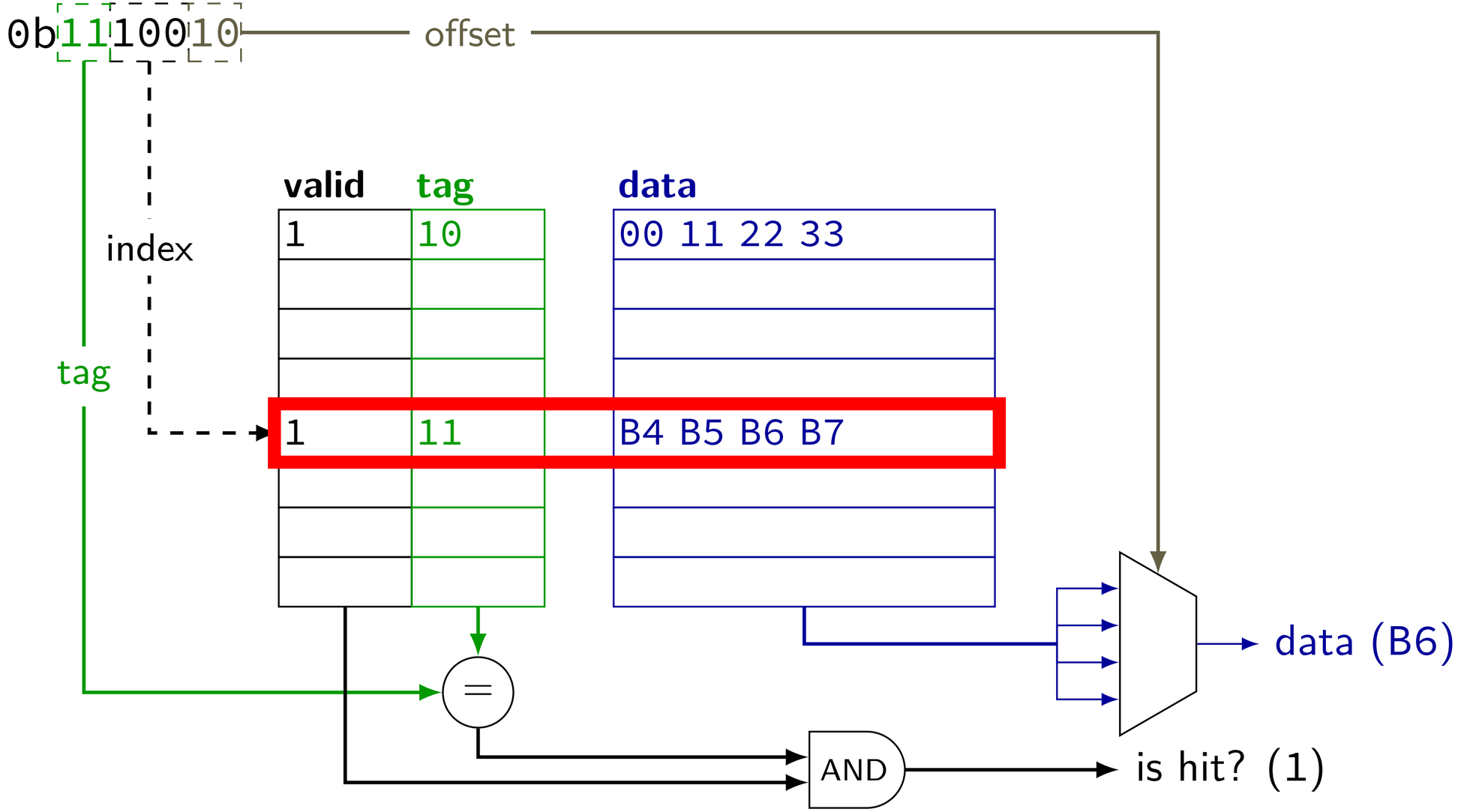
cache operation (read)



cache operation (read)



cache operation (read)



example access pattern (1)

example access pattern (1)

address (hex)	result
00000000 (00)	
00000001 (01)	
01100011 (63)	
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

2 byte blocks, 4 sets

index	valid	tag	value
00	0		
01	0		
10	0		
11	0		

example access pattern (1)

address (hex)	result
00000000 (00)	
00000001 (01)	
01100011 (63)	
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

2 byte blocks, 4 sets

index	valid	tag	value
00	0		
01	0		
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	
00000001 (01)	
01100011 (63)	
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	0		
01	0		
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	
01100011 (63)	
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

tag index offset

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	0		
10	0		
11	0		

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	0		
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	
01100010 (62)	
00000000 (00)	
01100100 (64)	

tag index offset

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	*1*	*01100*	*mem[0x62]* *mem[0x63]*
10	0		
11	0		

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	
00000000 (00)	
01100100 (64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	01100	mem[0x60] mem[0x61]
01	1	01100	mem[0x62] mem[0x63]
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	
01100100 (64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	01100	mem[0x60] mem[0x61]
01	1	01100	mem[0x62] mem[0x63]
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	miss
01100100 (64)	

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10	0		
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	miss
01100100 (64)	miss

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10	*1*	*01100*	*mem[0x64]* *mem[0x65]*
11	0		

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	miss
01100100 (64)	miss

tag index offset

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10	1	01100	mem[0x64] mem[0x65]
11	0		

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	miss
01100100 (64)	miss

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10	1	01100	mem[0x64] mem[0x65]
11	0		

miss caused by *conflict*

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits

example access pattern (1)

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	miss
01100100 (64)	miss

tag index offset

2 byte blocks, 4 sets

index	valid	tag	value
00	1	00000	mem[0x00] mem[0x01]
01	1	01100	mem[0x62] mem[0x63]
10	1	01100	mem[0x64] mem[0x65]
11	0		

miss caused by *conflict*

$m = 8$ bit addresses
$S = 4 = 2^s$ sets
$s = 2$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 5$ tag bits